

KUGGLE 2차 프로젝트

---

# 관람객 특성 기반 개별 영화 추천 시스템

이수아 이서경 권진서 진선우 홍승아

# 목차

Chapter 1. 서론

Chapter 2. 데이터 분석 (EDA)

Chapter 3. 데이터 전처리

Chapter 4. 모델링

Chapter 5. 모델 검증 및 성능평가

Chapter 6. 실무적 고도화

Chapter 7. 결론

chapter 1.

서론

# 프로젝트 배경 및 목표

## 배경

수많은 영화와 콘텐츠가 쏟아지는 환경 속,  
데이터 기반의 사용자 개별 맞춤형 영화 추천  
시스템의 필요성 증대

## 목표

사용자가 과거 매긴 영화의 평점을 바탕으로  
사용자의 취향을 분석해 사용자에게 유사한 영화를  
추천해주는 시스템 구축

추천 시스템의 핵심 가치:

사용자 자신도 좋아하지 몰랐던 취향을 시스템이 발견하고  
콘텐츠를 추천해주는 것



# 사용데이터

<https://www.kaggle.com/datasets/abhikjha/movielens-100k>

## MovieLens 100k

Small data to build movie recommender systems

---

columns

---

01

**movies.csv**

movieId, title, genres

**ratings.csv**

userId, movieId, rating, timestamp

**tags.csv**

userId, movieId, tag, timestamp

---

chapter 2.

# 데이터 분석 (EDA)

# 데이터 로드 및 상위 5개 행 미리보기

```
import pandas as pd
movies=pd.read_csv('movies.csv')
ratings=pd.read_csv('ratings.csv')
tags=pd.read_csv('tags.csv')
```

tags.head()

|   | userId | movieId | tag             | timestamp  |
|---|--------|---------|-----------------|------------|
| 0 | 2      | 60756   | funny           | 1445714994 |
| 1 | 2      | 60756   | Highly quotable | 1445714996 |
| 2 | 2      | 60756   | will ferrell    | 1445714992 |
| 3 | 2      | 89774   | Boxing story    | 1445715207 |
| 4 | 2      | 89774   | MMA             | 1445715200 |

ratings.head()

|   | userId | movieId | rating | timestamp |
|---|--------|---------|--------|-----------|
| 0 | 1      | 1       | 4.0    | 964982703 |
| 1 | 1      | 3       | 4.0    | 964981247 |
| 2 | 1      | 6       | 4.0    | 964982224 |
| 3 | 1      | 47      | 5.0    | 964983815 |
| 4 | 1      | 50      | 5.0    | 964982931 |

movies.head()

|   | movieId | title                              | genres                                      |
|---|---------|------------------------------------|---------------------------------------------|
| 0 | 1       | Toy Story (1995)                   | Adventure Animation Children Comedy Fantasy |
| 1 | 2       | Jumanji (1995)                     | Adventure Children Fantasy                  |
| 2 | 3       | Grumpier Old Men (1995)            | Comedy Romance                              |
| 3 | 4       | Waiting to Exhale (1995)           | Comedy Drama Romance                        |
| 4 | 5       | Father of the Bride Part II (1995) | Comedy                                      |

## 결측치 확인

```
print(f"Movies:\n{movies.isnull().sum()}\n")
print(f"Ratings:\n{ratings.isnull().sum()}\n")
print(f"Tags:\n{tags.isnull().sum()}\n")
```

```
Movies:
movieId    0
title      0
genres     0
dtype: int64
```

```
Ratings:
userId     0
movieId    0
rating     0
timestamp  0
dtype: int64
```

```
Tags:
userId     0
movieId    0
tag        0
timestamp  0
dtype: int64
```

→ 모든 칼럼에 결측치 존재 X

## 영화 ID(movieId) 중복 확인

```
print(f"movieId 중복 개수: {movies['movieId'].duplicated().sum()}")
movieId 중복 개수: 0
```

## 영화 제목(title) 중복 확인

→ ID는 다르지만 제목이 같은 영화 확인

```
duplicate_titles = movies[movies.duplicated(subset=['title'], keep=False)]
if len(duplicate_titles) > 0:
    print("\n--- 중복된 제목을 가진 영화 목록 ---")
    print(duplicate_titles.sort_values(by='title').head(5))
else:
    print("\n중복된 제목의 영화가 없습니다!")
```

```
--- 중복된 제목을 가진 영화 목록 ---
   movieId  title
4169    6003  Confessions of a Dangerous Mind (2002)
9106   144606  Confessions of a Dangerous Mind (2002)
650      838      Emma (1996)
5601   26958      Emma (1996)
5854   32600      Eros (2004)

   genres
4169  Comedy|Crime|Drama|Thriller
9106  Comedy|Crime|Drama|Romance|Thriller
650   Comedy|Drama|Romance
5601   Romance
5854   Drama
```

## 동일한 유저가 동일한 영화에남긴 중복 평점 확인

```
rating_dupes = ratings.duplicated(subset=['userId', 'movieId']).sum()
print(f"ratings 내 중복 데이터: {rating_dupes}개")
ratings 내 중복 데이터: 0개
```

## 평점의 최소 / 최대값 확인

→ 0.5~ 5.0 사이에 있으면 정상

```
print(ratings['rating'].describe()[['min', 'max']])

min    0.5
max    5.0
Name: rating, dtype: float64
```

## ratings에는 있는데 movies에는 없는 ID가 있는지 확인

```
movie_ids_in_ratings = set(ratings['movieId'].unique())
movie_ids_in_movies = set(movies['movieId'].unique())
missing_movies = movie_ids_in_ratings - movie_ids_in_movies
print(f"정보(제목 등)가 없는 영화 ID 개수: {len(missing_movies)}개")
정보(제목 등)가 없는 영화 ID 개수: 0개
```

## 제목(title)에서 연도(year) 추출

```
movies['year'] = movies['title'].str.extract(r'(\d{4})')
movies['year'] = movies['year'].str.extract(r'(\d{4})')
```



## 영화 태그(tag) 정규화

→ 정규화 후 발생한 중복 데이터 확인

```
tags['tag'] = tags['tag'].str.lower().str.strip()
duplicate_tags = tags.duplicated(subset=['userId', 'movieId', 'tag']).sum()
print(f"정규화 후 발견된 중복 태그: {duplicate_tags}개")
```

정규화 후 발견된 중복 태그: 0개

02

## 평점(ratings)과 영화 정보(movies) 결합

```
merged_df = pd.merge(ratings, movies,
                     on='movieId', how='inner')

print(merged_df.info())
print("\n--- 상위 5개 데이터 ---")
print(merged_df.head())
```

```
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   userId      100836 non-null int64
1   movieId     100836 non-null int64
2   rating      100836 non-null float64
3   timestamp   100836 non-null int64
4   title       100836 non-null object
5   genres      100836 non-null object
6   year        100818 non-null object
dtypes: float64(1), int64(3), object(3)
memory usage: 5.4+ MB
None
```

--- 상위 5개 데이터 ---

|   | userId | movieId | rating | timestamp | title                | genres                                            | year |
|---|--------|---------|--------|-----------|----------------------|---------------------------------------------------|------|
| 0 | 1      | 1       | 4.0    | 964982703 | Toy Story            | [Adventure, Animation, Children, Comedy, Fantasy] | 1995 |
| 1 | 1      | 3       | 4.0    | 964981247 | Grumpier Old Men     | [Comedy, Romance]                                 | 1995 |
| 2 | 1      | 6       | 4.0    | 964982224 | Heat                 | [Action, Crime, Thriller]                         | 1995 |
| 3 | 1      | 47      | 5.0    | 964983815 | Seven (a.k.a. Se7en) | [Mystery, Thriller]                               | 1995 |
| 4 | 1      | 50      | 5.0    | 964982931 | Usual Suspects, The  | [Crime, Mystery, Thriller]                        | 1995 |



## 최종 merged\_df의 결측치 확인

```
print("\n--- 결측치 확인 ---")  
print(merged_df.isnull().sum())
```

```
--- 결측치 확인 ---  
userId      0  
movieId     0  
rating      0  
timestamp   0  
title       0  
genres      0  
year        18  
dtype: int64
```

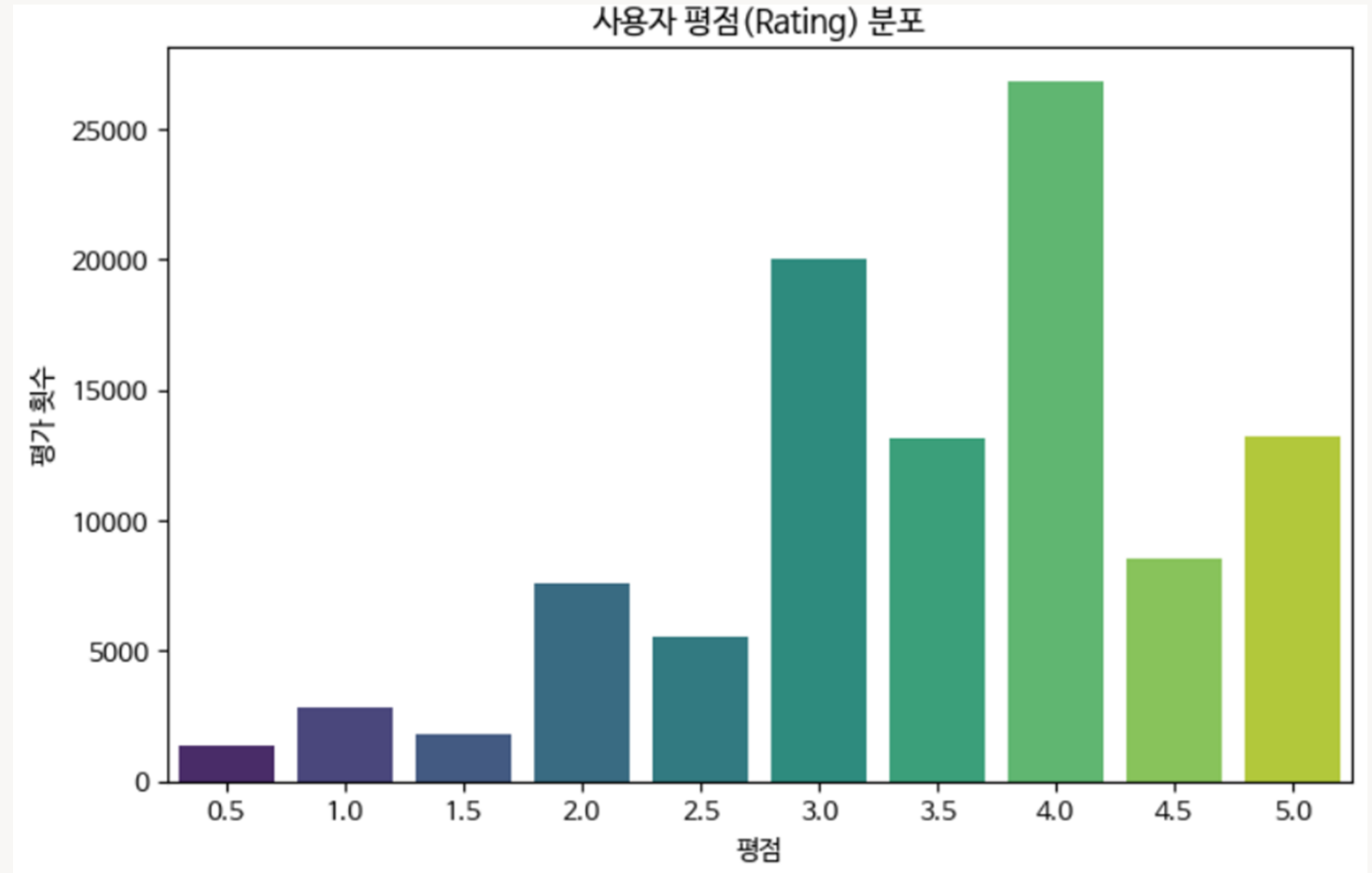
→ 결측치가 있는 year 칼럼에 대해 결측치 행 제거

```
merged_df = merged_df.dropna(subset=['year'])
```

## 평점(rating) 통계 요약 및 시각화

```
print("--- 평점 통계 요약 ---")
print(merged_df['rating'].describe())
```

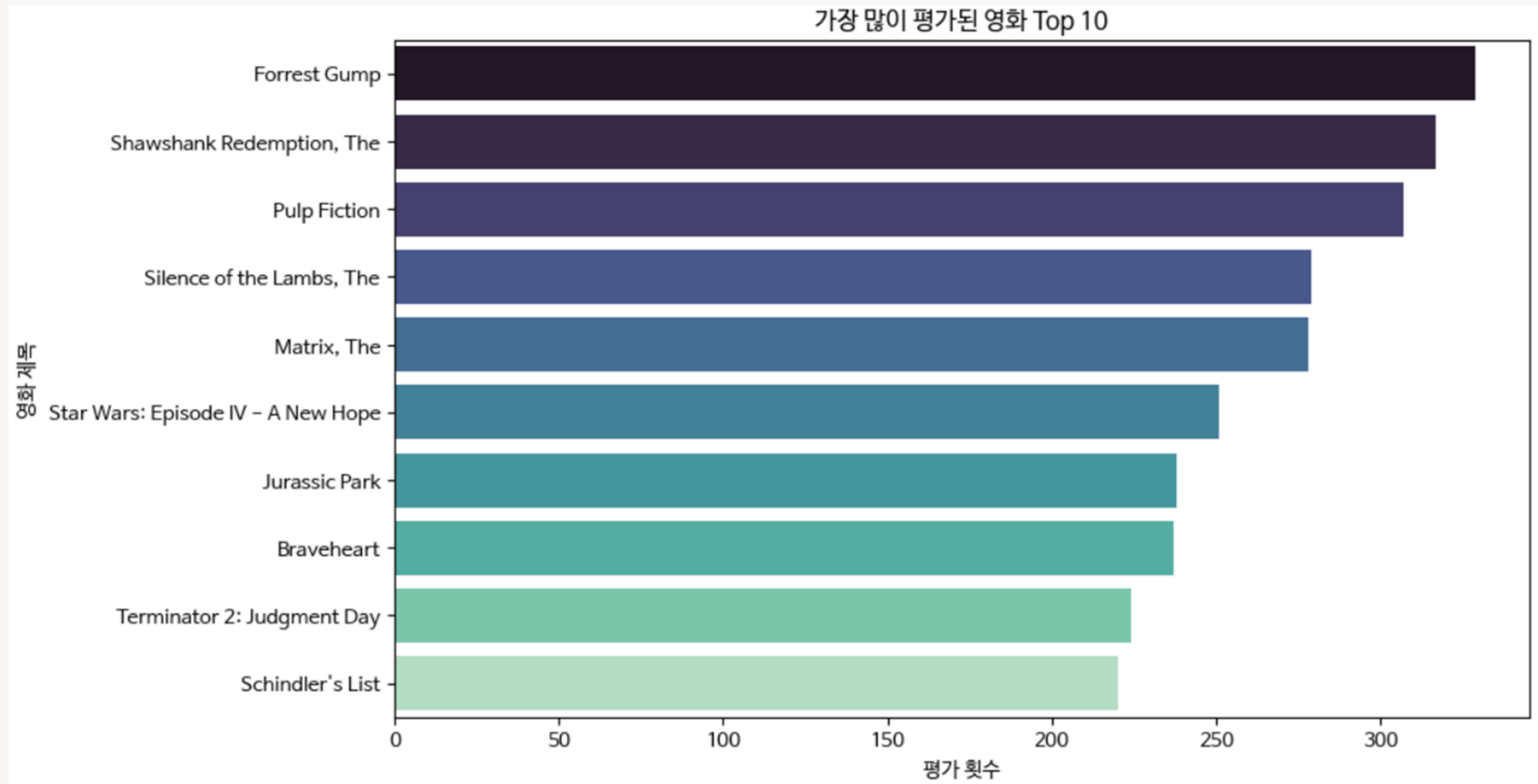
```
--- 평점 통계 요약 ---
count      100836.000000
mean         3.501557
std          1.042529
min          0.500000
25%          3.000000
50%          3.500000
75%          4.000000
max          5.000000
Name: rating, dtype: float64
```



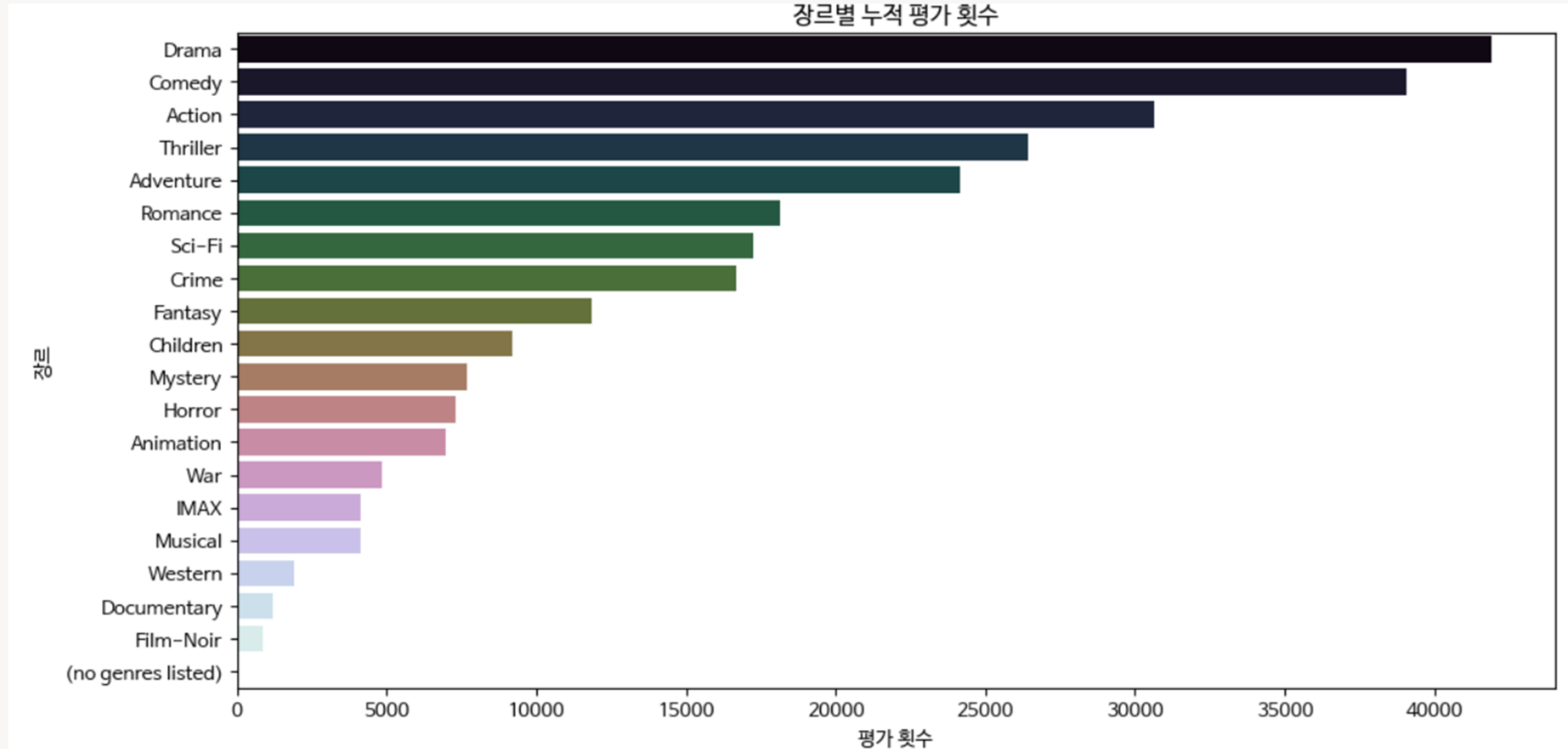
→ 대체로 높은 평점대를 보임



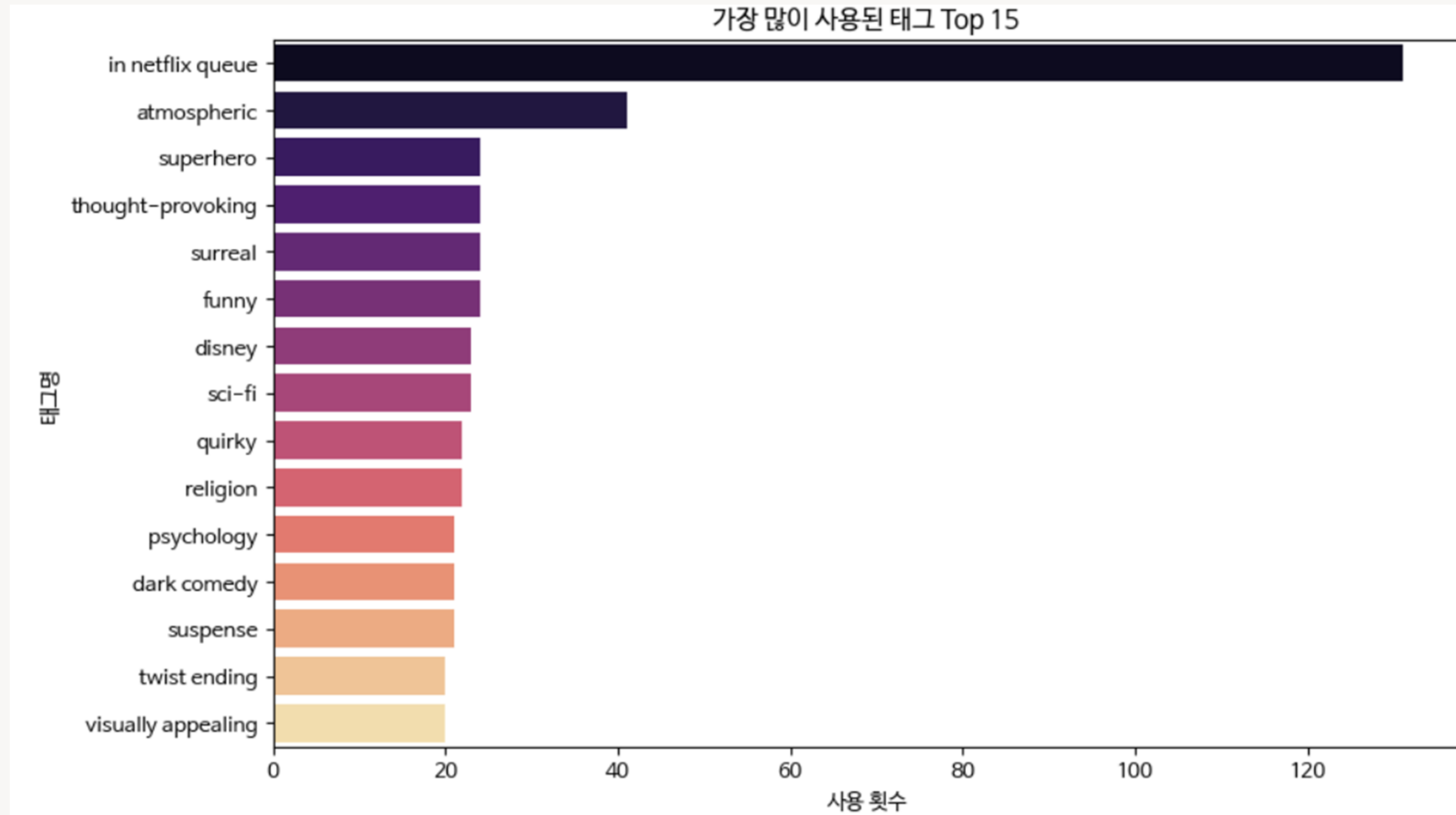
## 가장 많이 본(평가된) 영화



## 장르별 누적 평가 횟수 시각화



## 가장 많이 사용한 태그(tag)



→ 'in Netflix queue'는 넷플릭스 대기열에 있는 영화임을 말함.

chapter 3.

# 데이터 전처리

## 좋은 추천 시스템

→ 사용자가 무엇을 원하는지 빠르게 찾아내 사용자의 영화 선택의 즐거움이 배가 되도록 하는 것

추천 시스템은 크게 '콘텐츠 기반 필터링'과 '협업 필터링'으로 나뉘어진다.

---

01

### 콘텐츠 기반 필터링

사용자가 특정한 영화를  
매우 선호하는 경우,  
그 영화와 비슷한 콘텐츠를 가진  
다른 영화를 추천하는 방식

### 협업 필터링

사용자가 영화에 매긴 평점 정보나  
영화 시청 이력과 같은  
사용자 행동 양식만을  
기반으로 추천을 수행하는 방식



## 콘텐츠 기반 필터링 모델 제작을 위한 전처리

→장르(genre), 개봉연도(year), 태그(tag)를 분석하기 편하게 전처리한 후 하나의 컬럼으로 합치기

```
# movieId를 기준으로 중복을 제거하여 약 9,700개의 고유 영화 리스트 제작
movies = merged[['movieId', 'title', 'genres', 'year']].drop_duplicates(subset=['movieId']).reset_index(drop=True)

# genre 전처리: '|' 문자를 띄어쓰기(' ')로 변경
merged['genres_parsed'] = merged['genres'].str.replace('|', ' ', regex=False)

# year 전처리
merged['year_str'] = merged['year'].apply(lambda x: f"year_{int(x)}" if pd.notna(x) else "")

# tag 전처리: 영화별로 태그를 하나의 문자열로 묶기 (예: "funny Highly quotable will ferrell")
movie_tags = tags.groupby('movieId')['tag'].apply(lambda x: ' '.join(x.astype(str))).reset_index()
merged = pd.merge(merged, movie_tags, on='movieId', how='left')
```

```
# [핵심] 장르 + 태그 + 연도를 하나의 텍스트로 합치기
merged['combined_features'] = merged['genres_parsed'] + " " + merged['tag'] + " " + merged['year_str']

print("==전처리 후 데이터 확인==")
display(merged[['title', 'genres_parsed', 'combined_features']])
```

→ combined\_features에 영화에 대한 메타데이터가 모두 담기게 됨

| index | title                | genres_parsed                                               | combined_features                                                                         |
|-------|----------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 0     | Toy Story            | ['Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy'] | [Adventure, Animation, Children, Comedy, Fantasy] pixar pixar fun year_1995               |
| 1     | Grumpier Old Men     | ['Comedy', 'Romance']                                       | [Comedy, Romance] moldy old year_1995                                                     |
| 2     | Heat                 | ['Action', 'Crime', 'Thriller']                             | NaN                                                                                       |
| 3     | Seven (a.k.a. Se7en) | ['Mystery', 'Thriller']                                     | [Mystery, Thriller] mystery twist ending serial killer year_1995                          |
| 4     | Usual Suspects, The  | ['Crime', 'Mystery', 'Thriller']                            | [Crime, Mystery, Thriller] mindfuck suspense thriller tricky twist ending heist year_1995 |
| 5     | From Dusk Till Dawn  | ['Action', 'Comedy', 'Horror', 'Thriller']                  | NaN                                                                                       |

## 협업 필터링 모델 제작을 위한 전처리

→ 행: userid, 열: movieid, 값: rating 인 행렬 만들기

```
merged = pd.read_csv('merged_df.csv')
user_item_matrix = merged.pivot_table(index='userid', columns='movieid', values='rating')

# csv로 저장
output_filename = 'user_item_matrix.csv'
user_item_matrix.to_csv(output_filename)

# 행렬 만들기
shape = user_item_matrix.shape
head = user_item_matrix.head()

print(f"Shape: {shape}")
print(head.iloc[:5, :10])
```

```
Shape: (610, 9724)
movieid   1   2   3   4   5   6   7   8   9  10
userid
1         4.0 NaN 4.0 NaN NaN 4.0 NaN NaN NaN NaN
2         NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
3         NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
4         NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
5         4.0 NaN NaN NaN NaN NaN NaN NaN NaN NaN
```

→ 거의 대부분이 결측치(NaN)으로 구성된 희소행렬

chapter 4.

**모델링**



● 콘텐츠 기반 필터링 모델

TF/IDF 모델 적용

```
import ast
from sklearn.feature_extraction.text import TfidfVectorizer

# TF-IDF 모델 적용
# token_pattern 설정: 하이픈(-)을 단어의 일부로 인식하도록 정규표현식을 지정합니다.
tfidf = TfidfVectorizer(token_pattern=r'(?u)\b[^\w-]+\b')

# NaN 값을 빈 문자열로 대체하여 오류 방지
merged['combined_features'] = merged['combined_features'].fillna('')

# fit_transform을 통해 장르별 IDF 가중치를 계산하고 각 영화를 벡터로 변환
tfidf_matrix = tfidf.fit_transform(merged['combined_features'])

# 결과 확인 (DataFrame 형태로 변환하여 시각화)
feature_names = tfidf.get_feature_names_out()
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=feature_names,
    index=merged['title'])
```

→ 변환된 tfidf\_matrix는 대부분의 값이 0인 희소행렬.

각 컬럼: 학습된 단어(feature)

각 행: 영화

셀의 값: 해당 영화에서 단어의 TF-IDF 점수

```
tfidf_df.head()
```

|                            | 06  | 1900s | 1920s | 1950s | 1960s | 1970s | 1980s | 1990s | 2001-<br>like | 250 | ... |
|----------------------------|-----|-------|-------|-------|-------|-------|-------|-------|---------------|-----|-----|
| title                      |     |       |       |       |       |       |       |       |               |     |     |
| Toy Story                  | 0.0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0           | 0.0 | ... |
| Grumpier<br>Old Men        | 0.0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0           | 0.0 | ... |
| Heat                       | 0.0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0           | 0.0 | ... |
| Seven<br>(a.k.a.<br>Se7en) | 0.0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0           | 0.0 | ... |
| Usual<br>Suspects,<br>The  | 0.0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0           | 0.0 | ... |

5 rows × 1862 columns

→ 영화 데이터는 사용자가 모든 영화를 보지 않기 때문에 기본적으로 희소행렬일 수밖에 없다.

## ● 콘텐츠 기반 필터링 모델

### 코사인 유사도 계산 함수 작성

```
from sklearn.metrics.pairwise import cosine_similarity

def recommend_for_user_advanced(user_id, ratings_df=merged, movies_df=movies, tfidf_mat=tfidf_matrix, top_n=10):
    # 유저 시청 기록 확인
    user_history = ratings_df[ratings_df['userId'] == user_id]
    if user_history.empty:
        return f"유저 ID {user_id}의 기록이 없습니다.", None, None

    # 최고 평점, 최신 시청 영화 타겟팅
    top_movie_info = user_history.sort_values(by=['rating', 'timestamp'], ascending=[False, False]).iloc[0]
    target_movie_title = top_movie_info['title']
    target_rating = top_movie_info['rating']

    # 사용자가 이미 본 영화는 제외
    watched_movies = set(user_history['title'].tolist())

    # TF-IDF 벡터 추출
    try:
        idx = movies_df[movies_df['title'] == target_movie_title].index[0]
    except IndexError:
        return f"영화 '{target_movie_title}'를 찾을 수 없습니다.", None, None

    # 코사인 유사도 계산
    target_vector = tfidf_mat[idx]
    sim_scores_array = cosine_similarity([target_vector], tfidf_mat).flatten()
    sim_scores = list(enumerate(sim_scores_array))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    recommended_indices = []
```

```
for i, score in sim_scores:
    title = merged.iloc[i]['title']
    if title not in watched_movies:
        recommended_indices.append((i, score))
    if len(recommended_indices) == top_n:
        break
```

```
result_idx = [i[0] for i in recommended_indices]
```

```
# 결과 출력용: 제목, 장르, 결합된 특징(tag 등 확인용)
result_df = merged.iloc[result_idx][['title', 'genres_parsed', 'tag']].copy()
result_df['similarity'] = [score[1] for score in recommended_indices]

return target_movie_title, target_rating, result_df
```

→ return까지 전 과정이 def,  
recommend\_for\_user\_advanced에 딸려 있음

● 콘텐츠 기반 필터링 모델

모델 실행

```
# --- 테스트 실행 ---
t_i = int(input("유저 아이디를 입력하시오 : "))
top_movie, rating, recommendations = recommend_for_user_advanced(t_i, top_n=10)

if recommendations is not None:
    print(f"👤 [유저 {t_i}번 님을 위한 고도화된 맞춤 추천]")
    print(f"💡 기준 영화: '{top_movie}' (평점: {rating}점)")
    print(f"🎬 태그와 연도까지 분석하여 다음 영화들을 추천합니다!👤")
    print(recommendations.to_string(index=False))
```

→ 유저 아이디를 입력받아 모델 실행

유저 아이디를 입력하시오 : 26  
👤 [유저 26번 님을 위한 고도화된 맞춤 추천]  
💡 기준 영화: 'Firm, The' (평점: 4.0점)  
🎬 태그와 연도까지 분석하여 다음 영화들을 추천합니다!

→ 예시로 26번 유저의 경우 'Firm, The' 라는 영화를 기준으로  
이와 유사한 10가지 영화 추천

| title               | genres_parsed                                               | tag                       | similarity |
|---------------------|-------------------------------------------------------------|---------------------------|------------|
| Toy Story           | ['Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy'] | pixar pixar fun           | 0.0        |
| Grumpier Old Men    | ['Comedy', 'Romance']                                       | moldy old                 | 0.0        |
| Heat                | ['Action', 'Crime', 'Thriller']                             | NaN                       | 0.0        |
| Usual Suspects, The | ['Crime', 'Mystery', 'Thriller']                            | tricky twist ending heist | 0.0        |
| From Dusk Till Dawn | ['Action', 'Comedy', 'Horror', 'Thriller']                  | NaN                       | 0.0        |
| Bottle Rocket       | ['Adventure', 'Comedy', 'Crime', 'Romance']                 | off-beat comedy quirky    | 0.0        |
| Braveheart          | ['Action', 'Drama', 'War']                                  | sword fight Scotland      | 0.0        |
| Rob Roy             | ['Action', 'Drama', 'Romance', 'War']                       | NaN                       | 0.0        |
| Canadian Bacon      | ['Comedy', 'War']                                           | NaN                       | 0.0        |
| Desperado           | ['Action', 'Romance', 'Western']                            | NaN                       | 0.0        |

→ 일부 사용자들의 similarity(유사도)가 전부 0.0으로 나오는  
문제를 해결하기 위해, 협업 필터링 모델을 통해 보완

- 협업 필터링 모델

## SVD 알고리즘 학습

```
import pkg_resources
from surprise import Reader, Dataset, SVD

# 1. Surprise용 데이터 로드
reader = Reader(rating_scale=(0.5, 5.0))
data = Dataset.load_from_df(merged[['userId', 'movieId', 'rating']], reader)

# 2. 전체 데이터로 학습 데이터셋 생성
trainset = data.build_full_trainset()

# 3. SVD 알고리즘 적용
algo = SVD(n_factors=50, random_state=42)
algo.fit(trainset)
```

→ 이 알고리즘을 실행하면 merged 데이터로부터  
직접 SVD 알고리즘을 실행해 결과 저장



## • 협업 필터링 모델

### SVD 알고리즘 뜯어보기

```
import numpy as np
from scipy.sparse.linalg import svds

user_item_matrix = pd.read_csv("user_item_matrix.csv")

# 각 평점에서 평점 평균만큼 제거해 상대적인 선호도로 변환
R = user_item_matrix.values
user_ratings_mean = np.nanmean(R, axis=1)
R_demeaned = np.nan_to_num(R - user_ratings_mean.reshape(-1, 1))

# SVD 수행 (50개의 잠재 요인)
k = 50
U, sigma, Vt = svds(R_demeaned, k=k)

# 특이값 행렬(Sigma)을 대각행렬로 만들고 제곱근(sqrt)을 구하기
sigma_diag = np.diag(sigma)
sigma_sqrt = np.sqrt(sigma_diag)

# [핵심] 유저 & 영화 잠재 요인 행렬 분리
# U_factors: 유저가 50개의 요인을 각각 얼마나 좋아하는지 (User x 50)
user_factors = np.dot(U, sigma_sqrt)
# V_factors: 영화가 50개의 요인을 각각 얼마나 가지고 있는지 (Movie x 50)
# (Vt는 50 x Movie 형태이므로, 다루기 쉽게 Transpose(T)하여 Movie x 50으로 바꿉니다)
item_factors = np.dot(sigma_sqrt, Vt).T
```

```
# 예상 평점 도출 및 Dataframe 반환 (U*V는 예상 평점)
all_user_predicted_ratings = np.dot(user_factors, item_factors.T) + user_ratings_mean.reshape(-1, 1)

preds_df = pd.DataFrame(all_user_predicted_ratings, columns=user_item_matrix.columns,
                        index=user_item_matrix.index)
```

display(preds\_df)

|     | userId     | 1        | 2         | 3         | 4         | 5         | 6         | 7         | 8         | 9         | ... |
|-----|------------|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|
| 0   | 1.009983   | 4.044894 | 4.267660  | 4.159385  | 4.357601  | 4.310278  | 4.399713  | 4.209312  | 4.348167  | 4.312995  | ... |
| 1   | 2.000565   | 3.908544 | 3.885647  | 3.878968  | 3.881805  | 3.884946  | 3.873426  | 3.860747  | 3.879298  | 3.888571  | ... |
| 2   | 3.007609   | 2.544467 | 2.458986  | 2.447226  | 2.448348  | 2.467065  | 2.496004  | 2.513176  | 2.455442  | 2.465102  | ... |
| 3   | 3.996393   | 3.233774 | 3.363265  | 3.481274  | 3.544919  | 3.483388  | 3.469287  | 3.786904  | 3.549882  | 3.595796  | ... |
| 4   | 4.998831   | 3.701537 | 3.551346  | 3.697080  | 3.662268  | 3.674172  | 3.626022  | 3.616137  | 3.666586  | 3.665309  | ... |
| ... | ...        | ...      | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ... |
| 605 | 605.941718 | 3.621304 | 3.497116  | 3.655954  | 4.137745  | 3.875426  | 3.946726  | 3.962200  | 4.092382  | 4.214204  | ... |
| 606 | 607.035171 | 6.482122 | 6.355031  | 6.250846  | 6.984865  | 6.473166  | 5.938547  | 5.918922  | 6.821853  | 6.901435  | ... |
| 607 | 608.030954 | 2.398064 | 2.565520  | 3.398996  | 3.765720  | 3.371241  | 4.033089  | 3.352174  | 3.820687  | 3.729316  | ... |
| 608 | 608.999953 | 6.860812 | 21.790579 | 15.794690 | 19.280431 | 20.491460 | 19.628704 | 19.949063 | 19.852958 | 19.478128 | ... |
| 609 | 609.908569 | 4.802418 | 3.488716  | 3.891380  | 4.089612  | 3.763262  | 4.318236  | 4.095303  | 4.036392  | 4.291272  | ... |

610 rows × 9725 columns

→ 유저가 아직 보지 않은 영화에 대한 평점 예측 완료

## ● 하이브리드 모델

### 하이브리드 추천 모델 함수 정의

```
def get_advanced_hybrid_recommendations(user_id, top_n=10, cf_weight=0.7, cbf_weight=0.3):
    """
    1단계(Retrieval): SVD(CF)를 통해 사용자가 좋아할 만한 후보군 추출
    2단계(Ranking): 타겟 영화와의 콘텐츠 유사도(CBF)를 가중합하여 최종 순위 결정
    """

    # 1. 유저의 시청 기록 및 가장 선호하는(평점 높은) 영화 추출
    user_history = merged[merged['userId'] == user_id]
    if user_history.empty:
        return f"유저 ID {user_id}의 기록이 없습니다.", None

    # 유저가 5점을 준 영화 중 가장 최근 영화를 기준(Anchor)으로 설정
    topRated_movies = user_history.sort_values(by=['rating', 'timestamp'], ascending=False)
    target_movie_title = topRated_movies.iloc[0]['title']
    target_movie_id = topRated_movies.iloc[0]['movieId']
    watched_movie_ids = set(user_history['movieId'].tolist())

    # 2. CF 단계: SVD 모델을 이용해 '모든 영화'에 대한 예상 평점 계산 (Retrieval)
    all_movie_ids = movies['movieId'].unique()
    cf_predictions = []

    for m_id in all_movie_ids:
        if m_id not in watched_movie_ids:
            # Surprise SVD 모델 활용
            est_rating = algo.predict(user_id, m_id).est
            cf_predictions.append((m_id, est_rating))

    # 예상 평점 상위 100개를 후보군으로 선정
    candidate_df = pd.DataFrame(cf_predictions, columns=['movieId', 'cf_rating'])
    candidate_df = candidate_df.sort_values(by='cf_rating', ascending=False).head(100)

    # 3. CBF 단계: 기준 영화(target_movie)와 후보군 간의 유사도 계산 (Ranking)
    try:
        target_idx = movies[movies['movieId'] == target_movie_id].index[0]
        target_vector = tfidf_matrix[target_idx]

        # 후보군 영화들의 인덱스 추출
        candidate_indices = movies[movies['movieId'].isin(candidate_df['movieId'])].index
        candidate_vectors = tfidf_matrix[candidate_indices]

        # 코사인 유사도 계산
        sim_scores = cosine_similarity(target_vector, candidate_vectors).flatten()

        # 유사도 점수 매핑
        sim_map = {movies.iloc[idx]['movieId']: score for idx, score in zip(candidate_indices, sim_scores)}
        candidate_df['cbf_similarity'] = candidate_df['movieId'].map(sim_map)

    except Exception as e:
        print(f"CBF 계산 중 오류 발생: {e}")
        candidate_df['cbf_similarity'] = 0

    candidate_df['cf_score_norm'] = (candidate_df['cf_rating'] - 0.5) / 4.5
    candidate_df['hybrid_score'] = (candidate_df['cf_score_norm'] * cf_weight) + \
        (candidate_df['cbf_similarity'] * cbf_weight)

    # 5. 최종 결과 정리
    final_recommendations = pd.merge(candidate_df, movies[['movieId', 'title', 'genres']], on='movieId')
    final_recommendations = final_recommendations.sort_values(by='hybrid_score', ascending=False).head(top_n)

    return target_movie_title, final_recommendations[['title', 'genres', 'cf_rating', 'cbf_similarity', 'hybrid_score']]
```

→ CBF는 콘텐츠 기반 필터링 모델, CF는 협업 필터링 모델을 가리킴

```
# 예상 평점 상위 100개를 후보군으로 선정
candidate_df = pd.DataFrame(cf_predictions, columns=['movieId', 'cf_rating'])
candidate_df = candidate_df.sort_values(by='cf_rating', ascending=False).head(100)

# 3. CBF 단계: 기준 영화(target_movie)와 후보군 간의 유사도 계산 (Ranking)
try:
    target_idx = movies[movies['movieId'] == target_movie_id].index[0]
    target_vector = tfidf_matrix[target_idx]

    # 후보군 영화들의 인덱스 추출
    candidate_indices = movies[movies['movieId'].isin(candidate_df['movieId'])].index
    candidate_vectors = tfidf_matrix[candidate_indices]

    # 코사인 유사도 계산
    sim_scores = cosine_similarity(target_vector, candidate_vectors).flatten()

    # 유사도 점수 매핑
    sim_map = {movies.iloc[idx]['movieId']: score for idx, score in zip(candidate_indices, sim_scores)}
    candidate_df['cbf_similarity'] = candidate_df['movieId'].map(sim_map)

except Exception as e:
    print(f"CBF 계산 중 오류 발생: {e}")
    candidate_df['cbf_similarity'] = 0

candidate_df['cf_score_norm'] = (candidate_df['cf_rating'] - 0.5) / 4.5
candidate_df['hybrid_score'] = (candidate_df['cf_score_norm'] * cf_weight) + \
    (candidate_df['cbf_similarity'] * cbf_weight)

# 5. 최종 결과 정리
final_recommendations = pd.merge(candidate_df, movies[['movieId', 'title', 'genres']], on='movieId')
final_recommendations = final_recommendations.sort_values(by='hybrid_score', ascending=False).head(top_n)

return target_movie_title, final_recommendations[['title', 'genres', 'cf_rating', 'cbf_similarity', 'hybrid_score']]
```

→ 협업 필터링으로 예상 평점 행렬 계산 후  
콘텐츠 기반 필터링으로 코사인 유사도 계산  
하이브리드 점수 결합을 통해 두 모델 결합

# ● 하이브리드 모델

## 모델실행

```
# --- 시스템 실행 ---
user_input = int(input("추천을 받을 유저 아이디를 입력하세요: "))
anchor_movie, result = get_advanced_hybrid_recommendations(user_input)

print(f"\n✅ [유저 {user_input}번 맞춤형 하이브리드 추천]")
print(f"🔴 기준 영화: {anchor_movie} (유저가 가장 선호하는 영화 기반)")
display(result)
```

→ 유저 아이디를 입력받아 하이브리드 모델 실행

추천을 받을 유저 아이디를 입력하세요: 26

✅ [유저 26번 맞춤형 하이브리드 추천]  
🔴 기준 영화: Firm, The (유저가 가장 선호하는 영화 기반)

|   | title                                             | genres                                       | cf_rating | cbf_similarity | hybrid_score |
|---|---------------------------------------------------|----------------------------------------------|-----------|----------------|--------------|
| 0 | Amelie (Fabuleux destin d'Amélie Poulain, Le)     | ['Comedy', 'Romance']                        | 4.193033  | 0.0            | 0.574472     |
| 1 | Shawshank Redemption, The                         | ['Crime', 'Drama']                           | 4.158892  | 0.0            | 0.569161     |
| 2 | Dr. Strangelove or: How I Learned to Stop Worr... | ['Comedy', 'War']                            | 4.146467  | 0.0            | 0.567228     |
| 3 | Wallace & Gromit: The Wrong Trousers              | ['Animation', 'Children', 'Comedy', 'Crime'] | 4.128822  | 0.0            | 0.564483     |
| 4 | Eternal Sunshine of the Spotless Mind             | ['Drama', 'Romance', 'Sci-Fi']               | 4.123287  | 0.0            | 0.563622     |
| 5 | Ghost in the Shell (Kôkaku kidôtai)               | ['Animation', 'Sci-Fi']                      | 4.094027  | 0.0            | 0.559071     |
| 6 | Blade Runner                                      | ['Action', 'Sci-Fi', 'Thriller']             | 4.088325  | 0.0            | 0.558184     |
| 7 | Streetcar Named Desire, A                         | ['Drama']                                    | 4.082606  | 0.0            | 0.557294     |
| 8 | Lawrence of Arabia                                | ['Adventure', 'Drama', 'War']                | 4.061997  | 0.0            | 0.554088     |
| 9 | Hoop Dreams                                       | ['Documentary']                              | 4.052674  | 0.0            | 0.552638     |

→ 이전의 콘텐츠 기반 필터링과 같이,  
26번 유저에 대해 영화 ‘Firm, The’를 기준으로  
하이브리드 점수가 높은 상위 10개 영화 추천

chapter 5.

# 모델 검증 및 성능 평가



## 성능 평가 함수 `evaluate_recommender` 정의

```
def evaluate_recommender(predictions, top_n=10):
    """
    nDCG@K: 랭킹 품질 진단 (0~1, 높을수록 좋음) [cite: 17, 41]
    Coverage: 추천의 다양성 및 쓸림 현상 진단 [cite: 18, 43]
    """

    user_est_true = defaultdict(list)
    for uid, iid, true_r, est, _ in predictions:
        user_est_true[uid].append((est, true_r, iid))

    all_ndcgs = []
    recommended_items = set()
    # 테스트 셋에 존재하는 모든 고유 아이템 추출
    all_unique_items = set([p[1] for p in predictions])

    for uid, user_ratings in user_est_true.items():
        # 예측 평점 기준 내림차순 정렬 (Ranking 단계) [cite: 7, 33]
        user_ratings.sort(key=lambda x: x[0], reverse=True)
        top_k = user_ratings[:top_n]

        for _, _, iid in top_k:
            recommended_items.add(iid)

    # nDCG 계산 [cite: 15, 41]
    dcg = sum([(2**true_r - 1) / np.log2(i + 2) for i, (est, true_r, iid) in enumerate(top_k)])
    ideal_ratings = sorted([true_r for _, true_r, _ in user_ratings], reverse=True)[:top_n]
    idcg = sum([(2**r - 1) / np.log2(i + 2) for i, r in enumerate(ideal_ratings)])
```

→ nDCG: 추천된 영화 목록의 순위 품질을 측정함.

예상 평점이 높은 영화가 실제 평점도 높을 경우 좋은 점수를 받으며,  
0에서 1 사이의 값을 가진다.

```
if idcg > 0:
    all_ndcgs.append(dcg / idcg)

avg_ndcg = np.mean(all_ndcgs)
coverage = len(recommended_items) / len(all_unique_items)

# 에러 방지를 위해 필요한 변수들을 모두 반환합니다.
return avg_ndcg, coverage, len(all_unique_items), len(recommended_items)
```

→ Coverage: 추천 시스템이 얼마나 다양한 영화를 추천하는지 나타냄

## 성능 평가 함수 `evaluate_recommender` 실행&결과 확인

```
# --- 검증 실행 ---
ndcg_score, coverage_score, total_count, rec_count = evaluate_recommender(predictions, top_n=10)

print(f"📊 [하이브리드 모델 정밀 진단 결과]")
print(f"✅ nDCG@10 : {ndcg_score:.4f} (랭킹 품질)" # 상위 배치의 정확도 [cite: 16, 41]
print(f"✅ Coverage : {coverage_score:.4f} (다양성)" # 전체 중 추천된 비율 [cite: 18, 43]
print(f"💡 분석: 테스트 아이템 {total_count}개 중 {rec_count}개가 추천에 활용되었습니다.")
```

```
# --- 검증 실행 ---
ndcg_score, coverage_score, total_count, rec_count = evaluate_recommender(predictions, top_n=10)

print(f"📊 [하이브리드 모델 정밀 진단 결과]")
print(f"✅ nDCG@10 : {ndcg_score:.4f} (랭킹 품질)" # 상위 배치의 정확도 [cite: 16, 41]
print(f"✅ Coverage : {coverage_score:.4f} (다양성)" # 전체 중 추천된 비율 [cite: 18, 43]
print(f"💡 분석: 테스트 아이템 {total_count}개 중 {rec_count}개가 추천에 활용되었습니다.")
```

```
📊 [하이브리드 모델 정밀 진단 결과]
✅ nDCG@10 : 0.7957 (랭킹 품질)
✅ Coverage : 0.2671 (다양성)
💡 분석: 테스트 아이템 5185개 중 1385개가 추천에 활용되었습니다.
```

→ nDCG = 0.7957, 이는 하이브리드 모델이 사용자에게 실제로 선호할 만한 영화들을 추천 목록의 상위에 효과적으로 배치하고 있다는 것

→ Coverage = 0.2671, 이 수치는 모델이 특정 인기 영화나 패턴에만 집중하여 추천하는 쓸림 현상이 있을 수 있음을 시사한다.

### 모델 검증을 통한 결론

nDCG = 0.7957이라는 높은 랭킹 품질을 통해, 사용자 만족도 측면에서 강력한 성능을 보임.  
 하지만, Coverage = 0.2671이라는 다양성 지표를 보면, 추천되는 영화의 폭이 다소 제한적임을 보임  
 따라서, 시스템의 목적에 따라 다양성의 확보가 중요하다면, Coverage를 높이는 방안도 고려해볼만 함.

chapter 6.

# 실무적 고도화

앞에서의 내용에 따라, 모델의 품질은 최대한 유지한 채, 후보군 수를 늘려 Coverage 지표 상승을 기대할 수 있는 방법으로 실무적 고도화 작업을 진행한다.

가중치 조정을 통해 최고의 성능을 나타낼 수 있게끔 적절히 두 모델을 혼합한다.

01

## 전략 1

1. 후보군을 상위 150개로 확대해  
다양성 지표 상승 기대
2. cbf\_weight를 0.3에서 0.4로 높여  
내가 봤던 영화와 비슷한 영화에 더 가중치 부여

## 전략 2

1. 후보군을 1,000개로 확장하여  
정답이 포함될 확률 극대화
2. 확장된 후보 내에서 cf 모델 점수와  
cbf 모델 점수를 5:5로 결합해  
정교하게 정렬

# 하이브리드 추천 모델 재작성 및 재실행

```
def get_optimized_hybrid_recommendations(user_id, top_n=10, cf_weight=0.6, cbf_weight=0.4):
```

→ cbf\_weight를 높여(0.3 → 0.4) 개별 유저의 콘텐츠 취향 반영 강화 (Coverage 개선 유도)

```
candidate_df = pd.DataFrame(candidates, columns=['movieId', 'cf_rating'])
candidate_df = candidate_df.sort_values(by='cf_rating', ascending=False).head(150)
```

→ SVD 기반 후보군을 150개로 확장해 다양성 지표 상승 기대

```
# [4단계] Hybrid Scoring (정규화 및 가중합)
# CF 점수 정규화 (0.5~5.0 scale -> 0~1)
candidate_df['cf_norm'] = (candidate_df['cf_rating'] - 0.5) / 4.5

# 최종 하이브리드 점수 산출
candidate_df['final_score'] = (candidate_df['cf_norm'] * cf_weight) + \
                             (candidate_df['cbf_sim'] * cbf_weight)

# [5단계] 결과 반환
result = pd.merge(candidate_df, movies[['movieId', 'title', 'genres']], on='movieId')
result = result.sort_values(by='final_score', ascending=False).head(top_n)

return anchor_movie_title, result[['title', 'genres', 'cf_rating', 'cbf_sim', 'final_score']]
```

→ 기존의 점수에 모델별 가중치를 곱해 final\_score 생성

[최적화 모델 실행 완료]

기준 영화: Firm, The

|     | title                                          | genres                                   | cf_rating | cbf_sim  | final_score |
|-----|------------------------------------------------|------------------------------------------|-----------|----------|-------------|
| 88  | Reservoir Dogs                                 | ['Crime', 'Mystery', 'Thriller']         | 3.887861  | 0.273031 | 0.560927    |
| 15  | Fight Club                                     | ['Action', 'Crime', 'Drama', 'Thriller'] | 4.020452  | 0.210314 | 0.553519    |
| 0   | Amelie (Fabuleux destin d'Amélie Poulain, Le)  | ['Comedy', 'Romance']                    | 4.193033  | 0.060669 | 0.516672    |
| 127 | No Country for Old Men                         | ['Crime', 'Drama']                       | 3.835438  | 0.143474 | 0.502115    |
| 1   | Shawshank Redemption, The                      | ['Crime', 'Drama']                       | 4.158892  | 0.033137 | 0.501107    |
| 16  | Goodfellas                                     | ['Crime', 'Drama']                       | 4.015704  | 0.075649 | 0.499020    |
| 13  | Usual Suspects, The                            | ['Crime', 'Mystery', 'Thriller']         | 4.032203  | 0.068103 | 0.498202    |
| 18  | Star Wars: Episode IV - A New Hope             | ['Action', 'Adventure', 'Sci-Fi']        | 4.009777  | 0.074592 | 0.497807    |
| 11  | Star Wars: Episode V - The Empire Strikes Back | ['Action', 'Adventure', 'Sci-Fi']        | 4.042559  | 0.060213 | 0.496427    |
| 6   | Blade Runner                                   | ['Action', 'Sci-Fi', 'Thriller']         | 4.088325  | 0.042610 | 0.495487    |

→ 26번 유저에 대해 영화 ‘Firm, The’를 기준으로 final\_score가 높은 상위 10개 영화 추천



## 하이브리드 추천 모델 검증

```
# nDCG 계산: 관련 아이템이 상위에 랭킹될수록 높은 점수 부여 [cite: 84]
dcg = 0
for i, m_id in enumerate(current_rec_ids):
    rel = actual_rel_map.get(m_id, 0)
    dcg += (2**rel - 1) / np.log2(i + 2)

# IDCG: 실제 평점 기반 최적의 순서일 때의 점수 [cite: 84]
user_ratings = sorted(actual_rel_map.values(), reverse=True)[:top_n]
idcg = sum([(2**r - 1) / np.log2(i + 2) for i, r in enumerate(user_ratings)])

if idcg > 0:
    all_ndcgs.append(dcg / idcg)

avg_ndcg = np.mean(all_ndcgs) if all_ndcgs else 0
coverage = len(recommended_items) / len(total_test_items)
```

→ 기존 하이브리드 모델과는 달리, 테스트셋 기준이 아닌 모든 고유 영화 ID를 대상으로 Coverage 계산

```
# --- 실행 및 결과 출력 ---
# 실무 환경과 유사하게 50명의 유저를 샘플링하여 진단 [cite: 59, 91]
test_users = merged['userId'].unique()[:50]
ndcg_val, cov_val, rec_count, total_count = evaluate_optimized_hybrid_fixed(test_users)

print(f"🔗 [수정된 모델 정밀 진단 결과]")
print(f"✅ nDCG@10 : {ndcg_val:.4f} (랭킹 품질)")
print(f"✅ Coverage : {cov_val:.4f} (추천 다양성)")
print(f"💡 분석: 테스트 아이템 {total_count}개 중 {rec_count}개가 추천에 활용되었습니다.")
```

```
🔗 [수정된 모델 정밀 진단 결과]
✅ nDCG@10 : 0.0431 (랭킹 품질)
✅ Coverage : 0.0129 (추천 다양성)
💡 분석: 테스트 아이템 9724개 중 125개가 추천에 활용되었습니다.
```

→ nDCG = 0.0431 라는 점은, 매우 낮은 수준의 랭킹 품질을 보여줌

→ Coverage = 0.0129 라는 점은, 테스트 데이터셋에 존재하는 9724개의 고유 아이템 중 약 1.29%에 해당하는 125개의 아이템을 추천에 활용했음을 의미

### 전략 1 모델 검증을 통한 결론

출력 결과(nDCG = 0.0431, Coverage = 0.0129) 는 이 모델이 매우 낮은 랭킹 품질과 다양성을 보임  
이는 최적화된 모델이 실제로 성능 저하를 가져왔고, 따라서 최종적으로 **채택할 수 없음**을 나타냄

# 하이브리드 추천 모델 재작성 및 재실행

```
# 후보군을 기존 150개에서 1,000개로 대폭 확장하여 Recall 확보
candidate_df = candidate_df.sort_values(by='cf_rating', ascending=False).head(1000)
```

→ Retrieval(Stage 1): 후보군을 1,000개로 확장하여  
정답이 포함될 확률(Recall) 극대화

```
# --- 실행부 ---
user_id_to_test = 26
anchor_title, rec_list = get_final_hybrid_recommendations(user_id=user_id_to_test)

print(f"🚀 [후보군 1,000개 확장 모델 실행]")
print(f"🌟 기준 영화: {anchor_title}")
display(rec_list)
```

🚀 [후보군 1,000개 확장 모델 실행]

🌟 기준 영화: Firm, The

|   | movieid | title                                             | genres                                       | cf_rating | cbf_sim | final_score |
|---|---------|---------------------------------------------------|----------------------------------------------|-----------|---------|-------------|
| 0 | 4973    | Amelie (Fabuleux destin d'Amélie Poulain, Le)     | ['Comedy', 'Romance']                        | 4.193033  | 0.0     | 0.410337    |
| 1 | 318     | Shawshank Redemption, The                         | ['Crime', 'Drama']                           | 4.158892  | 0.0     | 0.406544    |
| 2 | 750     | Dr. Strangelove or: How I Learned to Stop Worr... | ['Comedy', 'War']                            | 4.146467  | 0.0     | 0.405163    |
| 3 | 1148    | Wallace & Gromit: The Wrong Trousers              | ['Animation', 'Children', 'Comedy', 'Crime'] | 4.128822  | 0.0     | 0.403202    |
| 4 | 7361    | Eternal Sunshine of the Spotless Mind             | ['Drama', 'Romance', 'Sci-Fi']               | 4.123287  | 0.0     | 0.402587    |
| 5 | 741     | Ghost in the Shell (Kôkaku kidôtai)               | ['Animation', 'Sci-Fi']                      | 4.094027  | 0.0     | 0.399336    |
| 6 | 541     | Blade Runner                                      | ['Action', 'Sci-Fi', 'Thriller']             | 4.088325  | 0.0     | 0.398703    |
| 7 | 1104    | Streetcar Named Desire, A                         | ['Drama']                                    | 4.082606  | 0.0     | 0.398067    |
| 8 | 1204    | Lawrence of Arabia                                | ['Adventure', 'Drama', 'War']                | 4.061997  | 0.0     | 0.395777    |
| 9 | 246     | Hoop Dreams                                       | ['Documentary']                              | 4.052674  | 0.0     | 0.394742    |

→ 26번 유저에 대해 영화 ‘Firm, The’를 기준으로  
final\_score가 높은 상위 10개 영화 추천



## 하이브리드 추천 모델 검증

```
# 1. nDCG@K 계산: 관련 아이템이 상위에 위치할수록 가중치 부여
dcg = 0
for i, m_id in enumerate(current_rec_ids):
    rel = actual_rel_map.get(m_id, 0) # 정답셋에 없으면 0점
    dcg += (2**rel - 1) / np.log2(i + 2)
```

```
# IDCG: 실제 평점 기반 최적의 순서일 때의 점수
user_ratings = sorted(actual_rel_map.values(), reverse=True)[:top_n]
idcg = sum([(2**r - 1) / np.log2(i + 2) for i, r in enumerate(user_ratings)])

if idcg > 0:
    all_ndcgs.append(dcg / idcg)
```

```
# 2. 최종 지표 산출
avg_ndcg = np.mean(all_ndcgs) if all_ndcgs else 0
coverage = len(recommended_items) / len(total_items_in_system)
```

→ 기존 하이브리드 모델과는 달리, 테스트셋 기준이 아닌 모든 고유 영화 ID를 대상으로 Coverage 계산

```
# --- 지표 측정 실행 ---
test_users = merged['userId'].unique()[:200]
ndcg_val, cov_val, rec_count, total_count = evaluate_final_hybrid(test_users)

print(f"🎯 [최종 고도화 모델 진단 결과]")
print(f"✅ nDCG@10 : {ndcg_val:.4f} (랭킹 품질)")
print(f"✅ Coverage : {cov_val:.4f} (추천 다양성)")
print(f"💡 분석: 시스템 전체 {total_count}개 영화 중 {rec_count}개가 추천에 활용되었습니다.")
```

```
🎯 [최종 고도화 모델 진단 결과]
✅ nDCG@10 : 0.0000 (랭킹 품질)
✅ Coverage : 0.0550 (추천 다양성)
💡 분석: 시스템 전체 9724개 영화 중 535개가 추천에 활용되었습니다.
```

→ nDCG = 0.0000 라는 점은, 매우 낮은 수준의 랭킹 품질을 보여줌

→ Coverage = 0.0550 라는 점은, 테스트 데이터셋에 존재하는 9724개의 고유 아이템 중 약 5.5%에 해당하는 535개의 아이템을 추천에 활용했음을 의미

### 전략 2 모델 검증을 통한 결론

출력 결과(nDCG = 0.0000, Coverage = 0.0550) 는 이 모델이 매우 낮은 랭킹 품질과 다양성을 보임  
이는 최적화된 모델이 실제로 성능 저하를 가져왔고, 따라서 최종적으로 **채택할 수 없음**을 나타냄

chapter 7.

**결론**

## 실무적 고도화를 통한 결론

---

후보군을 1,000개까지 늘려 다양성(Coverage)은 이전 대비 약 5배 개선( $0.0129 \rightarrow 0.0550$ )됨

하지만 9,724개의 전체 영화 중 유저가 실제 시청한 영화가 1,000개의 후보군 안에 포함되지 않아 nDCG가 0점으로 수렴하는 Recall(재현율) 실패 현상 나타남

실무 아키텍처인 Retrieval 및 Re-ranking으로 구성된 Two-Stage 구조는 대규모 데이터에서 효율적이지만, 현재의 데이터 규모와 특성상 Sparsity(희소성)가 너무 높아 좁은 후보군 내에서 정답을 찾기 매우 어려운 구조임이 판명

따라서, 후보군 확장 시도에도 불구하고 정답군과의 교집합 확보가 어려워  
실무적인 Two-Stage 구조보다는, 전체 아이템을 대상으로 정교하게 순위를 매기는  
이전 하이브리드 모델을 최종안으로 확정

---

**Q&A**

# 감사합니다

이수아 이서경 권진서 진선우 홍승아